

Lab05S01: GraphQL vs REST - Desenho e Preparação do Experimento

1. DESENHO DO EXPERIMENTO

A. Hipóteses Nula e Alternativa

Pergunta de Pesquisa 1 (RQ1): Respostas às consultas GraphQL são mais rápidas que respostas às consultas REST?

- **H₀ (Hipótese Nula):** Não existe diferença estatisticamente significativa no tempo de resposta entre APIs GraphQL e REST ($\mu_{\text{GraphQL}} = \mu_{\text{REST}}$)
- **H₁ (Hipótese Alternativa - Bilateral):** Existe diferença estatisticamente significativa no tempo de resposta entre APIs GraphQL e REST ($\mu_{\text{GraphQL}} \neq \mu_{\text{REST}}$)

Pergunta de Pesquisa 2 (RQ2): Respostas às consultas GraphQL tem tamanho menor que respostas às consultas REST?

- **H₀ (Hipótese Nula):** Não existe diferença estatisticamente significativa no tamanho da resposta entre GraphQL e REST ($\mu_{\text{tamanho_GraphQL}} = \mu_{\text{tamanho_REST}}$)
- **H₁ (Hipótese Alternativa - Unilateral):** O tamanho da resposta GraphQL é significativamente menor que REST ($\mu_{\text{tamanho_GraphQL}} < \mu_{\text{tamanho_REST}}$)

B. Variáveis Dependentes

VD1: Tempo de Resposta (Response Time)

- **Definição:** Intervalo de tempo entre o envio da requisição e o recebimento completo da resposta
- **Unidade:** Milissegundos (ms)
- **Método de Coleta:** Medição usando bibliotecas de timing (axios interceptors para REST; apollo-client para GraphQL)

VD2: Tamanho da Resposta (Payload Size)

- **Definição:** Tamanho total em bytes da resposta HTTP (content-length)
- **Unidade:** Kilobytes (KB)
- **Método de Coleta:** Headers HTTP content-length; captura de compressão GZIP

C. Variáveis Independentes

VI1: Tipo de API (Tratamento Principal)

- **Valores:** GraphQL, REST
- **Justificativa:** Foco principal do experimento

VI2: Complexidade da Consulta

- **Valores:**
 - **Simples:** Consulta de uma única tabela (ex: listar todos os usuários)
 - **Moderada:** Consulta com 1-2 níveis de relacionamento (ex: usuários + seus posts)
 - **Complexa:** Consulta com 3+ níveis de relacionamento (ex: usuários + posts + comentários + autores de comentários)
- **Justificativa:** Esperado que GraphQL apresente vantagens maiores em consultas complexas

VI3: Quantidade de Registros

- **Valores:** 100, 1.000, 10.000 registros
- **Justificativa:** Avaliar comportamento sob diferentes volumes de dados

VI4: Tipo de Campo Solicitado

- **Valores:**
 - **Todos os campos (Full):** REST retorna todos os campos; GraphQL solicita todos
 - **Campos selecionados (Partial):** REST retorna todos; GraphQL solicita apenas alguns (~50%)
- **Justificativa:** Avaliar vantagem de over-fetching avoidance do GraphQL

D. Tratamentos

Tratamento 1 (T1): Consulta Simples com Todos os Campos

- API: REST → GET /users | GraphQL → query listUsers { id, name, email, age, city, country }
- Registros: 100, 1.000, 10.000
- Medições por volume: 30 repetições

Tratamento 2 (T2): Consulta Simples com Campos Selecionados

- API: REST → GET /users (retorna todos) | GraphQL → query listUsers { id, name, email }
- Registros: 100, 1.000, 10.000
- Medições por volume: 30 repetições

Tratamento 3 (T3): Consulta Moderada com Todos os Campos

- API: REST → múltiplas chamadas (GET /users + GET /posts) | GraphQL → query única
- Registros: 100, 1.000, 10.000

- Medições por volume: 30 repetições

Tratamento 4 (T4): Consulta Moderada com Campos Seleccionados

- API: REST → múltiplas chamadas com overhead | GraphQL → query com seleção parcial
- Registros: 100, 1.000, 10.000
- Medições por volume: 30 repetições

Tratamento 5 (T5): Consulta Complexa com Todos os Campos

- API: REST → 4+ chamadas (users → posts → comments → comment_authors) | GraphQL → query única aninhada
- Registros: 100, 1.000, 10.000
- Medições por volume: 30 repetições

Tratamento 6 (T6): Consulta Complexa com Campos Seleccionados

- API: REST → 4+ chamadas | GraphQL → query com seleção parcial de ~50% dos campos
- Registros: 100, 1.000, 10.000
- Medições por volume: 30 repetições

E. Objetos Experimentais

Ambiente de Banco de Dados:

- **SGBD:** PostgreSQL 14+
- **Tamanho:** Dados sintéticos em três volumes (100, 1.000, 10.000 registros)
- **Schema:** 4 tabelas relacionadas (users, posts, comments, comment_authors)
- **Relacionamentos:**
 - users (1) ↔ (N) posts
 - posts (1) ↔ (N) comments
 - comments (1) ↔ (N) comment_authors (ou associação usuário-comentário)

APIs Implementadas:

- **REST API:** Node.js + Express.js
- **GraphQL API:** Node.js + Apollo Server
- **Banco:** PostgreSQL

Cliente de Teste:

- **Framework:** Node.js + Axios (para REST) e Apollo Client (para GraphQL)
- **Ferramentas:** Grafana k6 para load testing (opcional para Sprint 2)

F. Tipo de Projeto Experimental

Tipo: Experimento Fatorial Completo com Blocos

Fatores:

1. Tipo de API (2 níveis: REST, GraphQL)
2. Complexidade (3 níveis: Simples, Moderada, Complexa)
3. Quantidade de Registros (3 níveis: 100, 1K, 10K)
4. Seleção de Campos (2 níveis: Todos, Parcial)

Total de Combinações: $2 \times 3 \times 3 \times 2 = 36$ tratamentos

Design: Experimento entre-grupos (between-subjects) com medidas repetidas

- Cada tratamento será executado 30 vezes
- Total de medições: $36 \times 30 = 1.080$ observações

G. Quantidade de Medições

Medições por Tratamento:

- **Repetições:** 30 por tratamento
- **Justificativa:** Tamanho amostral suficiente para teste t de Student ($n \geq 30$ para TCL)
- **Total de Medições:** 1.080

Métricas Coletadas por Medição:

1. Timestamp de início (T_start)
2. Timestamp de fim (T_end)
3. Tempo de resposta (T_end - T_start em ms)
4. Tamanho da resposta (content-length em KB)
5. Status HTTP
6. Tipo de API (REST/GraphQL)
7. Complexidade da consulta
8. Número de registros
9. Campos solicitados (todos/parcial)

Formato de Saída: CSV com colunas:

```
timestamp, api_type, complexity, record_count, field_selection, response_time_ms, payload
```

H. Ameaças à Validade

Validade Interna:

Ameaça	Descrição	Mitigação
Aquecimento do cache	Cache de aplicação/BD distorcer medições	Limpar cache entre execuções; primeira execução descartada
Variação em recursos do sistema	Processos de background afetarem timing	Executar em ambiente isolado; desativar serviços desnecessários
Instrumentação	Código de coleta adicionar overhead	Usar bibliotecas otimizadas; medir overhead separadamente
Maturation	Fatiga de processador durante experimento	Pausas entre blocos de testes; monitorar CPU/memória
Selection bias	Dados de teste não representativos	Usar dados sintéticos realistas com distribuição variada

Validade Externa:

Ameaça	Descrição	Mitigação
Generalização para ambiente real	BD/rede não representam produção	Documentar ambiente; replicar com dados reais em Sprint 2
Generalization across APIs	Resultados específicos para Express/Apollo	Escolher frameworks populares; mencionar limitações
Interação Tratamento × Contexto	GraphQL/REST comportamento diferente c/ plugins	Usar configurações padrão; testar com/sem compressão

Validade de Construto:

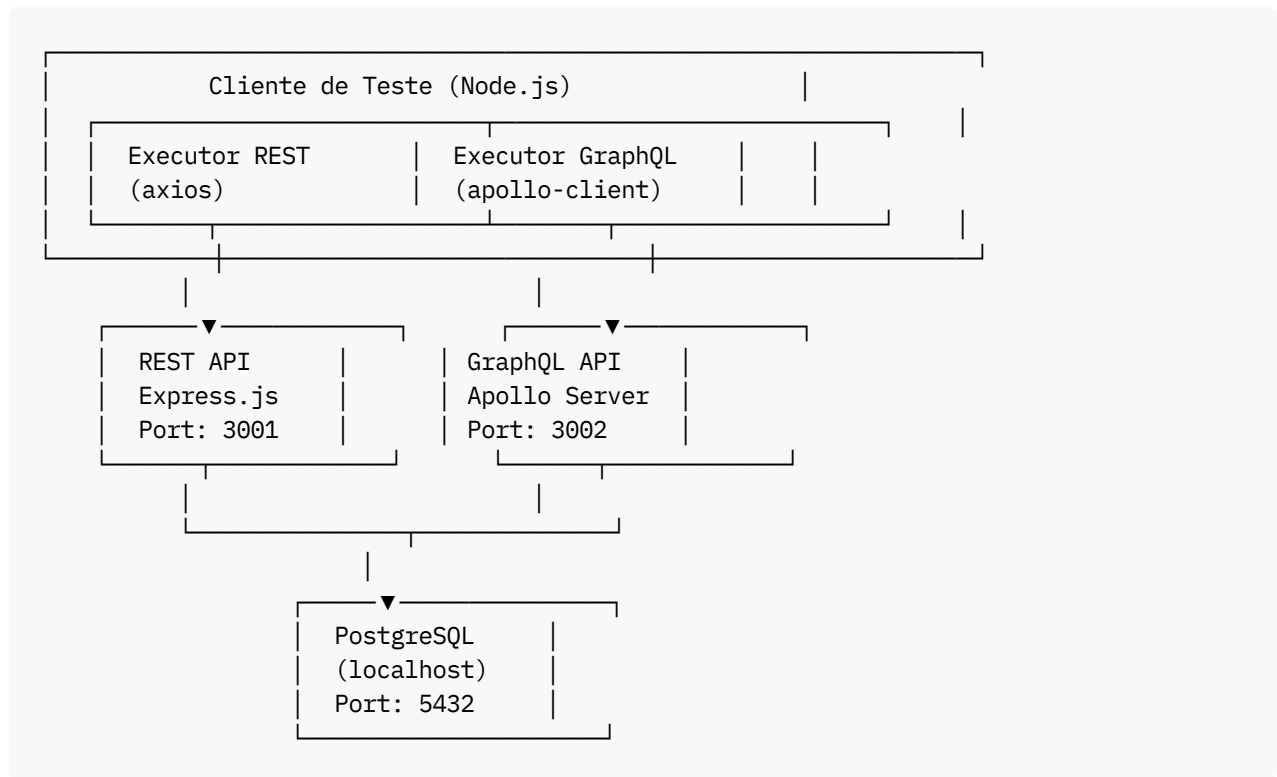
Ameaça	Descrição	Mitigação
Operacionalização inadequada de VD	Response time não captura experiência real	Incluir latência de network; testar com clientes remotos em Sprint 2
Sobre-simplificação de consultas	Queries muito básicas vs produção	Incluir paginação, filtros, argumentos em Sprint 2

Validade Estatística:

Ameaça	Descrição	Mitigação
Violação de normalidade	Dados não-normais invalidar t-test	Usar teste de Shapiro-Wilk; aplicar transformação log se necessário
Heterogeneidade de variância	Variância diferente entre grupos	Teste de Levene; usar Welch's t-test se necessário
Efeitos de ordem	Ordem de execução afetar resultados	Contrabalanceamento: alternar REST/GraphQL entre blocos

2. PREPARAÇÃO DO EXPERIMENTO

2.1 Arquitetura do Ambiente Experimental



2.2 Stack Tecnológico

Backend:

```
Node.js 18+
├── Express.js 4.x (REST)
├── Apollo Server 4.x (GraphQL)
└── pg 8.x (driver PostgreSQL)
```

Cliente de Teste:

```
Node.js 18+
├── axios (HTTP client REST)
├── @apollo/client (GraphQL client)
└── csv-parser (output de dados)
```

Banco de Dados:

```
PostgreSQL 14+
├── Dados sintéticos
└── Índices para performance
```

2.3 Schema do Banco de Dados

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  age INT,  
  city VARCHAR(50),  
  country VARCHAR(50),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE posts (  
  id SERIAL PRIMARY KEY,  
  user_id INT NOT NULL REFERENCES users(id),  
  title VARCHAR(200) NOT NULL,  
  content TEXT,  
  published_at TIMESTAMP,  
  view_count INT DEFAULT 0,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE comments (  
  id SERIAL PRIMARY KEY,  
  post_id INT NOT NULL REFERENCES posts(id),  
  content TEXT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE comment_authors (  
  id SERIAL PRIMARY KEY,  
  comment_id INT NOT NULL REFERENCES comments(id),  
  author_name VARCHAR(100),  
  author_email VARCHAR(100),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX idx_posts_user_id ON posts(user_id);  
CREATE INDEX idx_comments_post_id ON comments(post_id);  
CREATE INDEX idx_comment_authors_comment_id ON comment_authors(comment_id);
```

2.4 Estrutura de Diretórios

```
lab05s01/  
├── README.md  
├── package.json  
├── .env.example  
├──  
├── database/  
│   ├── schema.sql  
│   ├── seed_100.sql  
│   ├── seed_1000.sql  
│   ├── seed_10000.sql  
│   └── connection.js
```

```
├── rest-api/
│   ├── server.js
│   ├── routes/
│   │   ├── users.js
│   │   ├── posts.js
│   │   └── comments.js
│   └── middleware/
│       └── responseLogger.js
├── graphql-api/
│   ├── server.js
│   ├── schema.js
│   ├── resolvers/
│   │   ├── userResolver.js
│   │   ├── postResolver.js
│   │   └── commentResolver.js
│   └── middleware/
│       └── responseLogger.js
├── test-client/
│   ├── runner.js
│   ├── scenarios/
│   │   ├── scenario1_simple_all.js
│   │   ├── scenario2_simple_partial.js
│   │   ├── scenario3_moderate_all.js
│   │   ├── scenario4_moderate_partial.js
│   │   ├── scenario5_complex_all.js
│   │   └── scenario6_complex_partial.js
│   ├── utils/
│   │   ├── queryBuilder.js
│   │   ├── metricsCollector.js
│   │   └── csvWriter.js
│   └── results/
│       └── raw_results.csv
├── analysis/
│   ├── statistical_analysis.py
│   ├── requirements.txt
│   └── analysis.ipynb
└── docs/
    ├── experiment_design.md
    ├── setup_guide.md
    └── execution_plan.md
```

2.5 Scripts de Preparação

2.5.1 Seed de Dados (seed_100.sql)

```
-- Inserir 100 usuários
INSERT INTO users (name, email, age, city, country) VALUES
('Alice Johnson', 'alice.johnson@example.com', 28, 'São Paulo', 'Brazil'),
('Bob Smith', 'bob.smith@example.com', 35, 'Rio de Janeiro', 'Brazil'),
-- ... 98 registros adicionais (gerados programaticamente)
;

-- Inserir 200 posts (2 por usuário em média)
INSERT INTO posts (user_id, title, content, view_count) VALUES
(1, 'Post Title 1', 'Content of post 1', 150),
(1, 'Post Title 2', 'Content of post 2', 200),
-- ... 198 posts adicionais
;

-- Inserir 400 comentários
INSERT INTO comments (post_id, content) VALUES
(1, 'Great post!'),
(1, 'Thanks for sharing'),
-- ... 398 comentários adicionais
;

-- Inserir 400 autores de comentários
INSERT INTO comment_authors (comment_id, author_name, author_email) VALUES
(1, 'Comment Author 1', 'author1@example.com'),
(2, 'Comment Author 2', 'author2@example.com'),
-- ... 398 autores adicionais
;
```

2.5.2 REST API - Exemplo (users.js)

```
// rest-api/routes/users.js
const express = require('express');
const router = express.Router();
const pool = require('../database/connection');

router.get('/users', async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM users LIMIT 100');
    res.json(result.rows);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

router.get('/users/:id/posts', async (req, res) => {
  try {
    const result = await pool.query(
      'SELECT p.* FROM posts p WHERE p.user_id = $1',
      [req.params.id]
    );
    res.json(result.rows);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```

    res.status(500).json({ error: error.message });
  }
});

module.exports = router;

```

2.5.3 GraphQL API - Schema (schema.js)

```

// graphql-api/schema.js
const { gql } = require('apollo-server-express');

const typeDefs = gql`
  type User {
    id: Int!
    name: String!
    email: String!
    age: Int
    city: String
    country: String
    posts: [Post!]!
  }

  type Post {
    id: Int!
    title: String!
    content: String
    userId: Int!
    viewCount: Int
    comments: [Comment!]!
  }

  type Comment {
    id: Int!
    content: String!
    postId: Int!
    authors: [CommentAuthor!]!
  }

  type CommentAuthor {
    id: Int!
    name: String!
    email: String!
  }

  type Query {
    users(limit: Int): [User!]!
    userById(id: Int!): User
    posts(limit: Int): [Post!]!
    comments(limit: Int): [Comment!]!
  }
`;

module.exports = typeDefs;

```

2.5.4 Test Client - Runner (runner.js)

```
// test-client/runner.js
const axios = require('axios');
const { ApolloClient, gql, HttpLink, InMemoryCache } = require('@apollo/client');
const fs = require('fs');
const CSV = require('csv-string');

const REST_BASE_URL = 'http://localhost:3001';
const GRAPHQL_URL = 'http://localhost:3002/graphql';
const RESULTS_FILE = './results/raw_results.csv';
const REPETITIONS = 30;

const results = [];

async function measureREST(endpoint, description, recordCount) {
  const measurements = [];

  for (let i = 0; i < REPETITIONS; i++) {
    const startTime = Date.now();
    try {
      const response = await axios.get(endpoint);
      const endTime = Date.now();

      measurements.push({
        timestamp: new Date().toISOString(),
        api_type: 'REST',
        response_time_ms: endTime - startTime,
        payload_size_kb: JSON.stringify(response.data).length / 1024,
        http_status: response.status,
        query_description: description,
        record_count: recordCount
      });
    } catch (error) {
      console.error(`REST Error: ${error.message}`);
    }
  }

  return measurements;
}

async function measureGraphQL(query, description, recordCount) {
  const measurements = [];

  for (let i = 0; i < REPETITIONS; i++) {
    const startTime = Date.now();
    try {
      const client = new ApolloClient({
        ssrMode: true,
        link: new HttpLink({ uri: GRAPHQL_URL, credentials: 'same-origin' }),
        cache: new InMemoryCache(),
      });

      const response = await client.query({ query });
      const endTime = Date.now();
```

```

        measurements.push({
            timestamp: new Date().toISOString(),
            api_type: 'GraphQL',
            response_time_ms: endTime - startTime,
            payload_size_kb: JSON.stringify(response.data).length / 1024,
            http_status: 200,
            query_description: description,
            record_count: recordCount
        });
    } catch (error) {
        console.error(`GraphQL Error: ${error.message}`);
    }
}

return measurements;
}

async function runExperiment() {
    console.log('🚀 Iniciando experimento GraphQL vs REST...\n');

    // Scenario 1: Simple queries with all fields
    console.log('Scenario 1: Simple queries - All fields');
    results.push(... await measureREST(`${REST_BASE_URL}/users`, 'Simple-All-REST', 100));
    results.push(... await measureGraphQL(gql`{ users { id name email age city country } }`

    // ... mais cenários

    // Salvar resultados em CSV
    const csv = CSV.stringify([
        ['timestamp', 'api_type', 'response_time_ms', 'payload_size_kb', 'http_status', 'query_description', 'record_count'],
        ...results.map(r => [r.timestamp, r.api_type, r.response_time_ms, r.payload_size_kb, r.http_status, r.query_description, r.record_count])
    ]);

    fs.writeFileSync(RESULTS_FILE, csv);
    console.log(`✅ Resultados salvos em ${RESULTS_FILE}`);
}

runExperiment().catch(console.error);

```

2.6 Plano de Execução (Sprint 1 - Preparação)

Etapa	Descrição	Duração	Responsável
1	Preparar ambiente PostgreSQL + dados de seed	2h	DevOps/DB
2	Implementar REST API (Express.js)	4h	Backend
3	Implementar GraphQL API (Apollo)	4h	Backend
4	Implementar cliente de teste + coleta de métricas	3h	QA/Test
5	Validar configuração de ambiente	1h	Tech Lead
6	Documentar e preparar relatório	2h	Tech Lead

Total Sprint 1: ~16 horas

2.7 Dependências e Bibliotecas

```
{
  "dependencies": {
    "express": "^4.18.2",
    "apollo-server-express": "^4.11.0",
    "pg": "^8.10.0",
    "axios": "^1.6.0",
    "@apollo/client": "^3.8.0",
    "graphql": "^16.8.0",
    "dotenv": "^16.3.1",
    "csv-string": "^4.1.1"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

2.8 Checklist de Preparação

- ☐ PostgreSQL instalado e rodando
- ☐ Database criada e schema aplicado
- ☐ Dados de seed (100, 1K, 10K) carregados
- ☐ REST API implementada e testada (curl/Postman)
- ☐ GraphQL API implementada e testada (GraphQL Playground)
- ☐ Cliente de teste desenvolvido
- ☐ Coleta de métricas funcionando
- ☐ Diretório `results/` criado
- ☐ `.env` configurado corretamente
- ☐ Documentação finalizada
- ☐ Ambiente isolado e sem interferências

Conclusão

Este documento apresenta um design experimental robusto para avaliar quantitativamente os benefícios de GraphQL em relação a REST. A metodologia segue boas práticas de pesquisa empírica, com controle de variáveis, múltiplas repetições e mitigação de ameaças à validade. A preparação do experimento estabelece uma base sólida para execução confiável em Lab05S02 e análise estatística rigorosa em Lab05S03.

Responsáveis: [Nome do Grupo]

Data: November 27, 2025

Versão: 1.0

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20]

**

1. <https://smartdev.com/ai-powered-apis-grpc-vs-rest-vs-graphql/>
2. <http://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1668113>
3. <https://notionhive.com/blog/graphql-vs-rest-api-comparison>
4. <https://www.testscenario.com/api-performance-testing/>
5. <https://dl.acm.org/doi/pdf/10.1145/3702634.3702956>
6. https://acris.aalto.fi/ws/portalfiles/portal/83366076/computers_11_00065.pdf
7. <https://loadster.app/guides/api-performance-testing/>
8. <https://www.youtube.com/watch?v=uH0SxYdsjv4>
9. <https://hygraph.com/blog/graphql-vs-rest-apis>
10. <https://ai.stanford.edu/~ronnyk/GuideControlledExperiments.pdf>
11. https://www.reddit.com/r/graphql/comments/pom5h1/is_graphql_generally_worse_at_performance_than/
12. <https://www.diva-portal.org/smash/get/diva2:1449837/FULLTEXT01.pdf>
13. <https://zuplo.com/learning-center/api-performance-with-ab-testing>
14. <https://abstracta.us/blog/performance-testing/api-performance-testing/>
15. <https://mobilelive.ai/blog/graphql-vs-rest-what-you-didnt-know>
16. <https://research.aimultiple.com/graphql-vs-rest/>
17. <https://www.catchpoint.com/api-monitoring-tools/api-performance-testing>
18. <https://tailcall.run/graphql/graphql-vs-rest-api-comparison/>
19. <https://ph.pollub.pl/index.php/jcsi/article/download/7794/5149/36461>
20. <https://www.aptori.com/blog/api-performance-testing-best-practices-and-strategies>